

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación



Contenido

Introducción	04
¿Qué es una API?	04
REST: Representational State Transfer	05
Operaciones CRUD en REST	06
Métodos HTTP	06
Endpoints	07
Cómo probar las rutas o endpoints ?	08
Usando API REST con Flask + SQLAlchemy	09
Bibliografía	13

Clase 8



¡Te damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- Introducción a las API's Rest
- Concepto de API (analogías y ejemplos cotidianos)
- REST: arquitectura y características
- Operaciones CRUD en REST
- Métodos HTTP: definición y ejemplos
- Construcción de una API con Flask y SQLAlchemy
- Ejecución y pruebas con Postman

Introducción

En la clase anterior trabajamos con Flask en Python, aprendiendo a levantar un servidor web y manejar rutas básicas. Ahora vamos a dar un paso más: crear y consumir APIs REST.

Las APIs REST son fundamentales en el desarrollo moderno porque permiten que distintas aplicaciones (por ejemplo, una app móvil y un sistema web) puedan comunicarse entre sí de manera estandarizada. En esta clase veremos qué son las APIs, qué significa REST, cómo funcionan los métodos HTTP, qué son los endpoints y cómo probarlos con herramientas como Postman. Además, construiremos un ejemplo práctico con Flask + SQLAlchemy + SQLite.

¿Qué es una API?

Una API es un conjunto de reglas y protocolos que permite que dos aplicaciones o sistemas distintos se comuniquen entre sí. La API define qué se puede pedir, cómo se debe pedir y cómo responde el sistema.

Una API es como un mozo en un restaurante. El cliente pide un plato, el mozo lleva la orden a la cocina y luego devuelve la comida. El mozo (API) hace posible que el cliente (usuario) y la cocina (servidor) se entiendan.

Ejemplo de la vida real:

Cuando usás una app de clima, no descarga los datos del satélite directamente: consulta a una API meteorológica, que responde con los datos listos en JSON.

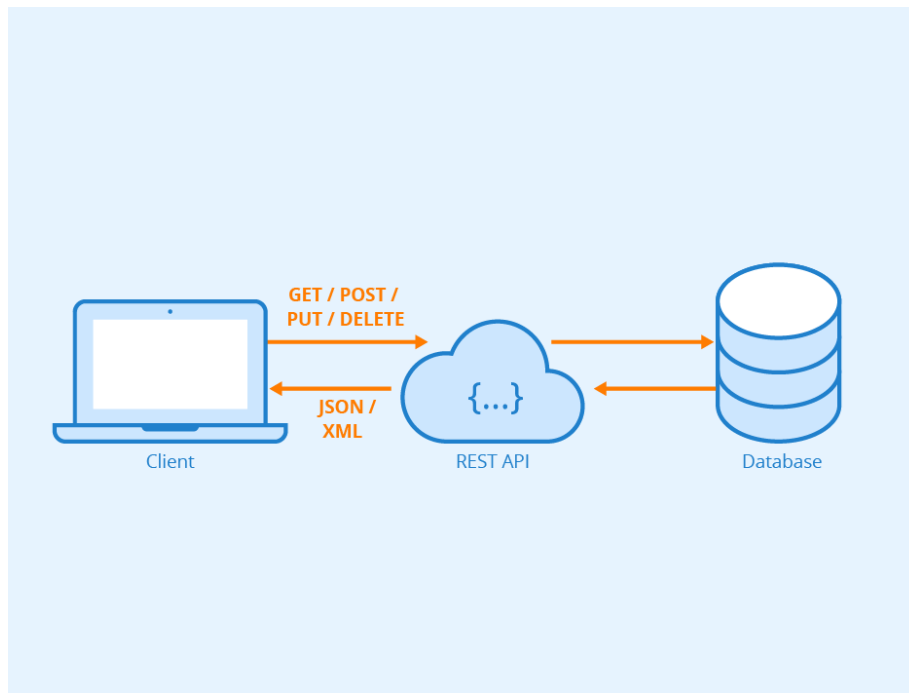


Imagen: Ilustración de interacción Cliente > Api > BBDD

REST: Representational State Transfer

REST es un estilo de arquitectura para diseñar APIs que aprovecha el protocolo HTTP.

Características principales:

- Basado en recursos: Todo se representa como un recurso, y cada recurso tiene una URL.

Ejemplo: /usuarios/15 representa al usuario con id=15.

- Uso de métodos HTTP estándar: GET, POST, PUT/PATCH, DELETE.
- Stateless: El servidor no guarda el "estado" de la sesión. Cada solicitud debe traer toda la información necesaria (como si cada vez fuese independiente).
- Representación flexible: Los recursos pueden enviarse en diferentes formatos (JSON es el más común).

REST busca simplicidad, escalabilidad y uso de estándares ya existentes en la web.

Operaciones CRUD en REST

Operaciones CRUD

CRUD es el acrónimo de las operaciones básicas sobre datos:

- C – Create (Crear): insertar un nuevo recurso.
- R – Read (Leer): consultar datos.
- U – Update (Actualizar): modificar un recurso existente.
- D – Delete (Eliminar): borrar un recurso.

En una API REST estas operaciones se implementan con los métodos HTTP:

Operación	Acción en API REST	Método HTTP
Create	Crear recurso	POST
Read	Leer recurso	GET
Update	Modificar recurso	PUT / PATCH
Delete	Eliminar recurso	DELETE

Métodos HTTP

Los métodos HTTP son las acciones que podemos pedirle al servidor.

- GET → recuperar información (lectura).

Ejemplo: GET /productos → devuelve lista de productos.

- POST → crear un recurso nuevo.

Ejemplo: POST /productos con un JSON {nombre: "Celular"} crea un producto.

- PUT → reemplazar un recurso existente.

Ejemplo: PUT /productos/3 → reemplaza todos los datos del producto con id=3.

- PATCH → actualizar parcialmente un recurso.

Ejemplo: PATCH /productos/3 → solo cambia el precio del producto 3.

- DELETE → eliminar un recurso.

Ejemplo: DELETE /productos/3 → elimina el producto con id=3.

Diferencia PUT vs PATCH:

PUT: reescribe el recurso entero.

PATCH: solo actualiza lo que se indique

Endpoints

Un endpoint es una URL específica dentro de la API que representa un recurso y sobre la que se pueden ejecutar acciones.

Ejemplo en una API de estudiantes:

- /estudiantes → lista o crea estudiantes.
- /estudiantes/5 → consulta, modifica o elimina al estudiante con id=5.

Cada endpoint + método HTTP define una acción concreta.

- GET /estudiantes → lista estudiantes.
- POST /estudiantes → agrega un estudiante.
- GET /estudiantes/5 → muestra estudiante con id=5.
- PUT /estudiantes/5 → actualiza estudiante con id=5.
- DELETE /estudiantes/5 → elimina estudiante con id=5.

¿Cómo probar las rutas o endpoints ?

Los **códigos de estado HTTP** son respuestas que el servidor devuelve al cliente (navegador, Postman, aplicación, etc.) cuando este realiza una solicitud. Estos códigos permiten saber si una solicitud fue exitosa o si hubo algún problema. Los códigos están clasificados en categorías según su primera cifra, y son fundamentales cuando se trabaja con APIs.

Códigos de Estado HTTP

Cada vez que el cliente hace una solicitud, el servidor responde con un código de estado HTTP, que indica cómo terminó la operación.

Principales categorías:

- 2xx – Éxito
- 200 OK: todo salió bien.
- 201 Created: recurso creado correctamente.
- 204 No Content: operación correcta, pero sin datos para devolver.
- 3xx – Redirección
- 301 Moved Permanently: el recurso se movió a otra URL.
- 4xx – Error del cliente
- 400 Bad Request: error en la solicitud (mal JSON, datos inválidos).
- 401 Unauthorized: falta autenticación.
- 404 Not Found: el recurso no existe.
- 5xx – Error del servidor
- 500 Internal Server Error: falla interna en el servidor.

Un 200 OK es como un "👍 todo bien".

Un 404 es como ir a una dirección y que te digan: "esa casa no existe".

Un 500 es cuando el mozo se cae y rompe el pedido antes de traértelo.

Recurso útil que lo explica con gatitos



[Link de ayuda](#)

Usando API REST con Flask + SQLAlchemy

Para probar APIs se utilizan herramientas como Postman o cURL.

Postman permite enviar solicitudes HTTP con diferentes métodos y ver la respuesta (códigos + datos en JSON).

Ejemplo:

- POST /estudiantes con { "nombre": "Ana", "edad": 20 }.
- GET /estudiantes para listar todos los estudiantes.

En Flask podemos usar SQLAlchemy + SQLite para guardar los datos y armar rutas que respondan a estas solicitudes.

Crearemos una API para gestionar estudiantes con **id, nombre y edad**.

Requisitos que necesitamos para crear nuestra primer API:

- Python 3.10+ instalado.
- Acceso a terminal (PowerShell en Windows / Terminal en macOS/Linux).
- Opcional: Postman para probarla.

Primero lo que haremos será crear el proyecto y el entorno virtual desde nuestra terminal y colocaremos lo siguiente:

Paso 1: Creamos nuestro entorno virtual (.venv)

```
PS D:\Python\Projects-In-Python\AyM_Python_DV\apiRestPython_DV> py -m venv .venv
>> .venv\Scripts\Activate
```

Paso 2: Instalamos las dependencias necesarias, creamos en el directorio de nuestro proyecto un archivo "requirements.txt" que contenga:

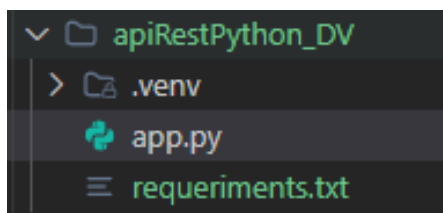
Flask==3.0.3

Flask-SQLAlchemy==3.1.1

Luego, instalamos las dependencias leyendo el archivo:

```
(.venv) PS D:\Python\Projects-In-Python\AyM_Python_DV\apiRestPython_DV> pip install -r requirements.txt
```

Paso 3: Deberíamos tener esta estructura



Paso 4: Código de ejemplo creación de la API:

Link de acceso al código completo:

<https://drive.google.com/file/d/1pun7R7nS9949pcjtZohK5qZMcconUkRA/view?usp=sharing>

Un error común al ejecutar este programa es que la versión de flask no soporte esta funcionalidad, pueden actualizarla usando:

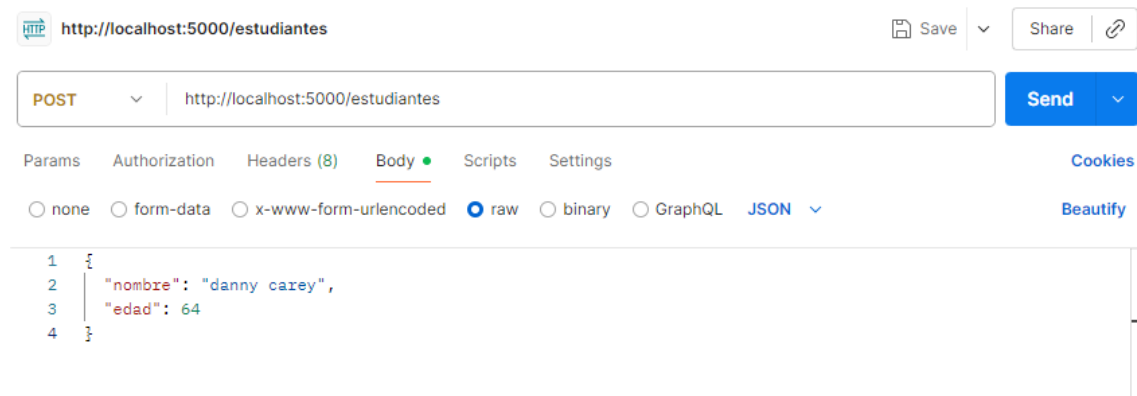
pip install --upgrade flask

En postman se probará de esta forma

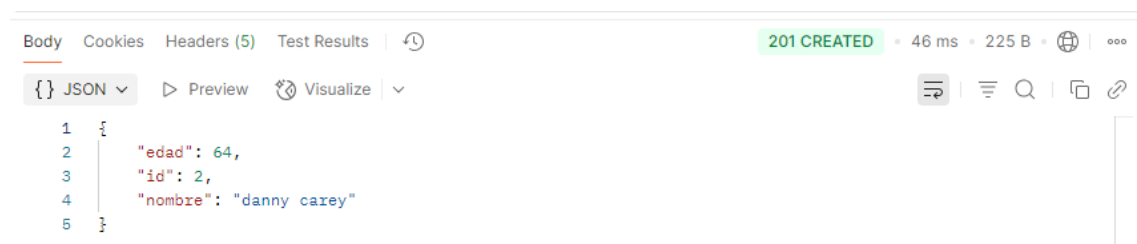
Levantamos el servidor con

python app.py

Una vez realizado podemos enviar solicitudes de diferentes formas.



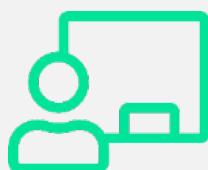
Si el servidor tiene una respuesta afirmativa tendremos esto como respuesta





Hemos llegado así al final de esta clase en la que vimos:

- Introducción a las API's Rest
- Concepto de API (analogías y ejemplos cotidianos)
- REST: arquitectura y características
- Operaciones CRUD en REST
- Métodos HTTP: definición y ejemplos
- Construcción de una API con Flask y SQLAlchemy
- Ejecución y pruebas con Postman



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.
¡Hasta la próxima clase!

Bibliografía

Mozilla Developer Network (MDN). *HTTP methods*.

<https://developer.mozilla.org/es/docs/Web/HTTP/Methods>

Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.

Grinberg, M. (2018). *Flask Web Development*. O'Reilly Media.

W3C. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*. <https://www.w3.org/TR/>